Computer Science Review 37 (2020) 100267

Contents lists available at ScienceDirect

Computer Science Review

journal homepage: www.elsevier.com/locate/cosrev

Survey

Motif discovery in networks: A survey

Shuo Yu, Yufan Feng, Da Zhang, Hayat Dino Bedru, Bo Xu *, Feng Xia

Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, School of Software, Dalian University of Technology, Dalian, 116620, China

ARTICLE INFO

Article history:
Received 8 October 2019
Received in revised form 12 April 2020
Accepted 15 May 2020
Available online xxxx

Keywords:
Motif discovery
Network motifs
Heterogeneous motifs
Motif visualization

ABSTRACT

Motifs are regarded as network blocks because motifs can be used to present fundamental patterns in networks. Motif discovery is well applied in various scientific problems, including subgraph mining and graph isomorphism tasks. This paper analyzes and summarizes current motif discovery algorithms in the field of network science with both efficiency and accuracy perspectives. In this paper, we present motif discovery algorithms, including MFinder, FanMod, Grochow, MODA, Kavosh, G-tries, QuateXelero, color-coding approaches, and GPU-based approaches. Based on that, we discuss the real-world applications of the algorithms mentioned above under different scenarios. Since motif discovery algorithms are diffusely demanded in many applications, several challenges may be firstly handled, including high computational complexity, higher order motif discovery, same motif detection, discovering heterogeneous sizes of motifs, as well as motif discovery results visualization. This work sheds light on current research progress and future research orientations.

© 2020 Published by Elsevier Inc.

Contents

1. Introduction 2
2. Preliminaries 2
   2.1. Network terminology 2
   2.2. Problem statement 2
3. Motif definitions and properties 3
4. Algorithms for motifs discovery 4
   4.1. MFinder 4
   4.2. FanMod 5
   4.3. Grochow 5
   4.4. MODA 6
   4.5. Kavosh 7
   4.6. G-tries 7
   4.7. QuateXelero 8
   4.8. Color-coding approach 8
   4.9. GPU-based approach 8
5. Motif discovery tools and platforms 9
   5.1. Motif discovery tools 10
      5.1.1. MFinder 10
      5.1.2. FanMod 10
      5.1.3. Regmex 10
      5.1.4. RSAT peak-motifs 10
      5.1.5. STAMP 10
      5.1.6. MotifNet 10
   5.2. Network analysis and visualization tools 10
      5.2.1. Gephi 10
      5.2.2. NodeXL 10

* Corresponding author.
E-mail address: boxu@dlut.edu.cn (B. Xu).

https://doi.org/10.1016/j.cosrev.2020.100267
1574-0137/© 2020 Published by Elsevier Inc.

5.2.3. Pajek 10
5.2.4. GUESS 10
6. Applications and challenges of motif discovery 10
7. Conclusion 11
Declaration of competing interest 11
Acknowledgments 11
References 11

1. Introduction

A network is generally used for modeling a system connecting various kinds of entities. Classifying nodes and predicting edges are two essential tasks that lots of complex systems tried to undertake [1,2]. Meanwhile, higher-order structures, as well as repeated small subgraph patterns, are also essential in a network [3]. These repeated subgraph patterns result in the generation of "motif". The concept of motifs is first proposed in the gene regulation network. Then, a considerable number of studies start to focus on the computational theory of network motifs [4,5]. Network motifs are regarded as network building blocks of complex networks, which is illustrated as "the patterns of connectivity that occur significantly more frequently than expected" [6]. Patterns between a small number of nodes in a network can be modeled by network motifs, which allows researchers "to discover" and "mine" interesting characteristics of "large-scale networks" by "using" motif discovery algorithms [7,8]. These motif discovery algorithms are generally applied in a biological domain. However, they can also be applied in other research fields.

Motif discovery algorithms are commonly applied to gene regulation networks, electronic circuits, brain neurons, etc. In social networks, motif discovery, especially motif triad, can also be applied in closure analysis. Similarly, modeling feedforward loops as motifs is fundamental in transcriptional regulation networks [9]. The frequency of motif is applied in describing the characteristics in random graph models [10]. In temporal networks with timestamp information, motif discovery is used to discover temporal paths [11,12].

Finding motifs is a computationally hard task due to the fact that matching graph patterns given to certain motif is always a NP-hard problem [13–16]. Additionally, with the ever-increasing sizes of motifs, the computational time grows exponentially. Therefore, in order to obtain results in a reasonable amount of time, various studies propose efficient motif discovery algorithms [17,18]. Nevertheless, the motif discovery task remains as a challenging problem requiring thorough network analysis, appropriate modeling strategy, and effective discovery algorithms. In this paper, we summarize the state-of-the-art strategies that are prevalently used in motif discovery. Though there exist several surveys sharing similar purposes, most of them focus on protein–protein networks in biology rather than pervasive motif discovery methods.

This survey specifically presents introduction of motif's formal definitions, elaboration of abundant motif discovery approaches, presentation of tools and methods, as well as discussion of motif discovery applications and challenges. The remainder of this paper is organized as follows. Section 2 gives a brief introduction of preliminaries of network motifs and describes the motif discovery problems. Section 3 introduces motif definitions as well as motif properties. Section 4 illustrates algorithms for motifs discovery, including MFinder, FanMod, etc. Section 5 presents the tools and platforms of motif discovery. Section 6 sets forth the applications and challenges of motif discovery. We conclude the paper in Section 7.

2. Preliminaries

In this section, we introduce some fundamental concepts in the network and give the formal definition of motif discovery.

2.1. Network terminology

Let $G = (V, E)$ be a network, wherein V is a set of vertices and E is a set of edges. In addition, $\mathcal{G}_r$ is a set containing r randomized networks, in which each randomized network has the same *degree distribution* as G. If G is a directed graph, then (u, v) represents a directed edge from u to v. We denote u as the *in-neighbor* of v, and v as the *out-neighbor* of u. In that case, the *in-degree* (*out-degree*) of v equals the number of its *in-neighbors* (*out-neighbors*). Also, the *bi-degree* of v is the number of its neighbor vertices, which contain both *in-neighbors* and *out-neighbors*. We define the vertices mentioned above as the *bi-neighbors* of v. Then, we define the neighbor set of v as $nbr(v)$, in which the vertices are the *in-neighbor* or *out-neighbor* of v.

We denote $g = (V_g, E_g)$ as the subgraph of G, i.e., $g \subseteq G$, if and only if there exists at least one bijective function ξ, which maps V_g to a subset $V_s \subseteq V$, satisfying that for any edge $(u, v) \in E_g$, there exists an edge $(\xi(u), \xi(v)) \in E$, and vice versa. The frequency of g in G equals to the total number of occurrences of g in G, which are also represented as $f(g, G)$. For a randomized network $H = (V_H, E_H)$, which is *degree-equivalent* of G, satisfying the two conditions depicted as follows: (i) $|V_H| = |V|$ and $|E_H| = |E|$ and (ii) there is a bijection ϕ: $V \to V_H$, such that for any vertex $v \in V$, the *in-degree*, *out-degree*, and *bi-degree* of v are the same as those of $\phi(v)$ in H.

In contrast to directed networks, for undirected networks, the *degree* of a node v is equal to the number of neighbor vertices. Furthermore, the *degree-equivalent* of undirected networks is different from that of directed networks in the condition (ii) mentioned above, the bijection of ϕ: $V \to V_H$, satisfying for any vertex v inV, v in G and $\phi(v)$ in H has the same *degree*.

2.2. Problem statement

Network motifs are over-presented subgraphs of real network comparing with randomized networks having the same degree distributions. In the process of detecting motifs in real networks, several randomized networks and a threshold are in need to evaluate the over-presented structures of networks. Hence, according to the definition of network motifs, *z-score* is applied in this situation. Moreover, we give a threshold α to restrict and evaluate the value of *z-score*.

Fig. 1 shows a real undirected network. Wherein, graph G is a real network, and $\mathcal{G}_2 = \{H_1, H_2\}$ is a set, which contains two randomized networks H_1 and H_2. We enumerate two subgraphs, i.e., (M_1 and M_2) in G, H_1 and H_2. Hence, $f(M_1, G) = 3$, meaning that the frequency of M_1 in G is 3, and $f(M_2, G) = 1$. Similarly, we can get $f(M_1, H_1) = 0$, $f(M_1, H_2) = 1$, $f(M_2, H_1) = 3$ and $f(M_2, H_2) = 2$. Then, we have $\bar{f}_{random}(M_1) = 1/2$ and $std(M_1) = 1/2$. The z-score of M_1 is 5. Similarly, for M_2, we have $\bar{f}_{random}(M_2) = 5/2$ and $std(M_2) = 1/2$. The *z-score* of M_1 equals -3.

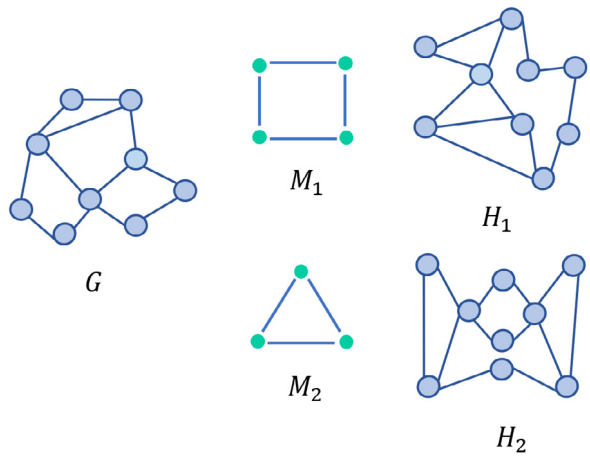

Fig. 1. A real undirected network G is modeled as the input graph. Two subgraphs M_1 and M_2 are chosen as the motif candidates. And the two randomized networks H_1 and H_2 are *degree-equivalent* to G.

In that case, the motif discovery process is described as follows.

Given $G, \mathcal{G}_r,\ r > 0, k > 2$ and $\alpha > 0$, the motif discovery problem enumerates all the k-motif of G in the randomized networks in $\mathcal{G}_k$.

3. Motif definitions and properties

In this section, we present the related definitions from different perspectives. Since motifs can characterize the dynamics and functional behaviors of networks, it can be applied to distinguish different types of networks based on motif statistics within the network. Scholars have discovered that there exist some particular patterns that can be modeled by certain network substructures in real-world networks. Such patterns have realistic meanings, including social relationships, protein complex, information infrastructure, etc. These patterns are reflected by some certain kinds of network substructures and appear much more often in real-world networks. In order to distinguish whether such kind of patterns are motifs or not, corresponding random networks are generated. Previously, graphlets are used to indicate structural patterns of networks, which encode diverse functional and relevant roles of different nodes. Based on that, statistically over-represented graphlets are defined as motifs [6,19]:

Definition 1. Network motifs are patterns of interconnections that occur with significantly higher frequencies in complex networks than those in random networks.

According to the definition, a motif is a subgraph that is over-represented over a threshold with more than a number of times by chance. A motif having k nodes is called k-motif. The smallest motifs occurring in directed networks contain two nodes [6]. Three-node motifs are the most frequent structures in undirected networks. When generating random networks, it is essential to make the incoming and outgoing node degrees similar to those in the real networks. Another condition is that when searching the k-motifs, it should be guaranteed that the frequencies of $(k-1)$-motifs of the original network and that of the randomized network are the same. This is because the larger motif with k nodes can be decomposed into at least one motif with $(k-1)$ nodes. It ensures that the high significance of a target motif does not simply generate from its sub-motifs. The two criteria stated above maintain the intrinsic global and local properties of the original and randomized networks. We aim to ensure that the appearance of motifs in networks is not determined by the properties of the networks, but specific to the particular networks.

Definition 1 is one of the most widespread network motif definitions, which is also regarded as the standard definition of network motifs. However, this definition is proposed purely based on the topologies of networks and neglects the individual properties of different motifs. Therefore, due to the different types of networks, such as directed and undirected networks, we provide different definitions under different scenarios.

Besides over-represented substructures, under-represented substructures in the networks are also meaningful for describing the features of a network. Thus, an under-represented definition of network motifs, *anti-motif* is proposed [20] as the following:

Definition 2. Subnetworks that are significantly under-represented in the original networks are *anti-motifs*.

Considering the subnetworks that induced from the original networks, the notion of induced motif has been proposed. An induced motif is a subnetwork that is composed of all edges between its nodes in the original networks. Moreover, a strongly connected motif is a subnetwork, in which there is a path from any node to any other node [21].

In gene regulatory networks, motifs are defined as small, repeated, and evolutionary conserved subnetworks [22]. To be more specific, the transcription network [23] illustrates the motifs with specific functionality in determining gene expressions, i.e., generating the temporal expression programs and controlling the responses to fluctuate external signals. Meanwhile, motifs in protein–protein networks represent the structural corrections among internal patterns. The motifs in social networks depict the different connected entities such as persons or companies in online social media. Additionally, in traffic networks, cars and stations are represented as nodes. Also, in labeled networks, the nodes may be labeled by more than one label from a finite set C, whose elements have semantic meanings in the real world. We can represent various labels by using different colors. Therefore, the definition of *colored-motifs* in [24] is shown in Definition 3:

Definition 3. A motif is a multi-set of elements from the set C of colors.

Besides the definitions of motif mentioned above, *maximal-motif* [25] is proposed, in which the edges and nodes cannot be added. The *maximal-motif* does not contain other subgraphs as motifs so that it can diminish the total number of discovered motifs. In comparison with randomized networks, three constraints are defined when classifying the motifs, that is, a probability threshold P, a uniqueness threshold U, and a minimum difference threshold D [17].

- A probability threshold P is defined to confine the probability that the frequency of a certain motif appears on a randomized network is greater than that of a real network. The threshold P is determined by comparing it with a large number of randomized networks, ensuring that the motif is over-represented in the original network. P can be estimated by assuming a random null hypothesis and *z-scores*, for a given network G_k the definition of $z\text{-}score(G_k)$ is described as in Eq. (1),

$$z - score(G_k) = \frac{f_{ori} - \bar{f}_{rand}}{std(f_{rand})} \tag{1}$$

  Herein, f_{ori} is the frequency of a given motif in G_k, while $\bar{f}_{rand}$ is the frequency in a randomized network. Also, the $std(f_{rand})$ is the standard deviation of motif frequency in different randomized networks.

- The second constraint ensures that the motif frequency in the original network, f_{ori} is higher than a *uniqueness* threshold U. The threshold U defines a quantitative minimum value to establish significance.
- The last constraint is proposed to prevent the conditions that the target motif has a small difference between f_{ori} and $\bar{f}_{rand}$ as well as a narrow distribution in random networks. Hence, the constraint is that f_{ori} is far larger than $\bar{f}_{rand}$. This can be formalized as $f_{ori} - \bar{f}_{rand} > D \times \bar{f}_{rand}$. Wherein, D is the threshold used to be the limitation of the minimum differences between f_{ori} and $\bar{f}_{rand}$.

According to the three constraints, a formal definition of network motifs is given as follows:

Definition 4. In Graph G, a network motif refers to an induced subgraph G_k constrained by a set of parameters $\{P, U, D, N\}$. Wherein, P, U, and D are the parameters mentioned above, and N refers to the corresponding random networks. G_k satisfies the following three conditions:

1. Over-representation:

$$Prob(\bar{f}_{random}(G_k) > f_{original}(G_k)) \leq P \tag{2}$$

2. Minimum frequency:

$$f_{original}(G_k) \leq U \tag{3}$$

3. Minimum deviation:

$$f_{original}(G_k) - \bar{f}_{random}(G_k) > D \times \bar{f}_{random}(G_k) \tag{4}$$

Moreover, Milo et al. [6] assigned the parameter set $\{P, U, D, N\}$ as $\{0.01, 4, 0.1, 1000\}$. However, the values of the parameters are set depending on the different motifs we want.

Considering the static significance of network motifs, authors in [20] use the *abundance*($\triangle$) rather than *z-score* mentioned above to describe the representation level of certain motifs. The *abundance* of G_k is presented in Eq. (5),

$$\triangle(G_k) = \frac{f_{original} - \bar{f}_{random}}{f_{original} + \bar{f}_{random} + \varepsilon} \tag{5}$$

Wherein, ε is a constant, which is used to ensure *abundance* not be misleading large when the motif frequency is small.

Kashtan et al. [26] defined the notion of *concentration*(C) to discover the static significance of motifs in the network (see Eq. (6).) Here, $\sum f(G_k)$ is the sum number of all frequencies of k-motifs.

$$P = \sum \frac{f(G_k)}{\sum f(G_k)} \tag{6}$$

4. Algorithms for motifs discovery

In this section, we summarize the most currently used methods for network motif discovery and respectively present representative methods. Moreover, some of those motif discovery methods have been implemented on motif detection software tools for network analysis.

The primary network motif discovery methods follow the same process. For a given target motif, these methods first enumerate all the k-motifs from the original network and then calculate the k-motif statistics of the given network. The results are generally the frequencies of non-isomorphic k-motifs. Furthermore, some algorithms of randomized networks maintain the global properties of the original networks, e.g., *degree distributions*. And then, the k-motif statistics are calculated on the randomized networks. Finally, based on the comparison of the statistical motif frequencies between the original network and randomized networks, the statistical distributions between the certain motifs and the most over-represented motif in the original network are evaluated. To summarize, the network motif discovery processes contain the following three phases:

- Discover the subgraph which occurs in the input network and calculate its frequencies.
- Compare and determine the isomorphic subgraphs, and group them into a subgraph according to the topological properties.
- Find higher frequencies compared with randomized networks.

The processes of enumerating all the subgraphs are computationally expensive even if the input graph is small and sparse. Scholars have put their efforts on accelerating this process. Herein, we introduce some representative methods among those motif discovering algorithms. Table 1 shows the comparison of the motif discovery methods that appear in this paper. Generally, existing techniques for discovering network motifs can be classified into two categories, including network-centric and motif-centric. Network centric approaches enumerate all subgraphs in the whole network, while motif-centric approaches search for a single query graph and then map the frequencies instead of enumerating. Comparing with network-centric approaches, motif-centric approaches can take advantages of subgraph symmetries.

4.1. MFinder

MFinder implements a subgraph sampling algorithm in motif discovery [27,28]. The subgraph sampling algorithm samples the n-node subgraphs by appending the connected edges randomly until each subgraph having n nodes. Details of this method are described as follows. The sampling algorithm starts by picking an edge randomly from the input network. Then the sampling algorithm expands the subgraph iteratively by choosing random neighboring edges until the sampling subgraph contains n nodes. When picking a random edge, this method aims to choose an edge that can expand the subgraph size by one. Randomly, the candidate edge is chosen from a list of candidate edges. Finally, those sampled n-subgraphs contain n nodes, and the edges in each n-subgraph connect the nodes in the original networks. The iterations that add edges to the subgraphs until the subgraph contains k nodes are called ESA (Enumerate Subgraph Algorithm). The pseudocode is described in Algorithm 1.

Algorithm 1 *Edge Sampling*(G, k)

Require: A graph $G = (V, E)$ and an integer $2 \leq k \leq |V|$
Ensure: *size-k* subgraphs census of G

```
{u, v} ← random edge from E;
V' ← {u, v};
while |V'| ≠ k do
    {u, v} ← random edge between V' and N(V');
    V' ← V' ∪ u, v
end while
return
```

In this method, the authors discover the network motifs whose $z - scores$ are significantly high. In order to coincide with the enumerating process, Z-score is converted to $z - score = (C_{real} - C_{random})/\sigma_{rand}$. Here, C_{real} and C_{rand} are the concentrations in real and random networks, respectively. To get the concentrations of k-subgraphs, the authors calculate the probability P of enumerating the different k-subgraphs. The score of a k-subgraph i is initialized as $S_i = 0$, and the score can be computed iteratively

Table 1
Summary of common motif detection methods.

Algorithm	Years	Parallel	Directed	(Non-)Induced	Centric	MaxMotifSize
MFinder	2002	×	○	Induced	Network	4
FanMod	2006	×	○	Induced	Network	8
Grochow	2007	×	○	Both	Motif	∞
Color-coding	2008	×	×	Non-Induced	–	10
MODA	2009	×	○	Both	Motif	∞
Kavosh	2009	×	○	Induced	Network	∞
G-tries	2010	×	○	Induced	Network	∞
QuateXelero	2013	×	○	Induced	Network	∞
GPU-based	2017	○	○	Induced	Network	∞

as $S_i = S_i + \frac{1}{P}$. If there exist L subgraphs in the input network, the concentration of type i k-subgraph can be calculated in Eq. (7):

$$c_i = \frac{S_i}{\sum_{k=1}^{L} S_k} \tag{7}$$

One advantage of this algorithm is that running time is asymptotically independent of network size. Moreover, the algorithm can construct the network motif with a low concentration (e.g., *concentration* $= 10^{-5}$). However, the analysis of the time complexity of the MFinder has proved that it takes $O(n^n)$ for each sample of n-subgraph from the input networks. Besides, it does not take the "graph isomorphism" time cost into consideration. Moreover, this algorithm is inevitable to sample the same subgraph repetitively [29]. In conclusion, this method performs better than the methods that exhaustively enumerate network motifs. Further, it can only enumerate motifs up to the size of six nodes, which is limited by its running time.

4.2. FanMod

FanMod is a motif discovery tool that contains a refined motif sampling algorithm RANK-ESU [30,31]. The procedure of finding repetitive isomorphism motifs is evitable in FanMod. Instead of starting from an edge (as presented in MFinder), this method starts with a single "root" node and expands the subgraph recursively. Although this method uses the basic algorithm to enumerate all the k-subgraphs, a certain probability P_d is taken into consideration in each recursive enumerating procedure. The probability P_d is associated with the depth of the enumeration. Since each subgraph from the input graphs appears only once in the searching tree as well as the time-consuming complete traversal, FanMod explores parts of the graph when searching motifs. Specifically, each leaf is reached with the same probability when searching part of the searching tree. Therefore, at depth d, P_d is the probability of whether we traverse the searching subtree rooted from each child vertex. Corresponding to ESA, the operations of starting with a random node when sampling the k-subgraph is defined as ESU (Enumerate Subgraphs). The pseudocode of ESU is presented in Algorithm 2.

Algorithm 2 *Enumerate subgraphs*(G, k)*(ESU)*

Require: A graph $G = (V, E)$ and an integer $2 \le k \le |V|$
Ensure: *size-k* subgraphs of G
1: **for** each node $v \in V$ **do**
2: $V_{ext} = \{u \in N(\{v\}) : u > u\}$
3: *ExtSubgraph*$(\{v\}, V_{ext}, v)$
4: **end for**
5: **return**

FanMod arranges all the nodes in the input networks in ascending order. Therefore, the nodes with higher indices will be appended into the subgraph. A set of possible vertices V_{ext} for an extension is maintained. And each node in V_{ext} is an exclusive neighborhood $N_{excl}(v, V_{Subgraph})$, where the $V_{Subgraph}$ is the node-set of the sampling subgraph. Therefore, $N_{excl}(v, V_{Subgraph})$ includes the neighborhood nodes which are from $V \backslash V_{Subgraph}$ and adjacent to at least one node in $V_{Subgraph}$. The node is removed from the V_{ext} as long as the node is chosen for an extension. Furthermore, the exclusive neighbors of the sampling subgraph are added to the new possible extension set. This procedure is recursively executed until the sampling subgraph identifies the target k-subgraph. The algorithm described above guarantees that each target sampling subgraph is enumerated exactly at once, as the neighbors that are not exclusive will be added to another sampling motif. FanMod uses a highly efficient third-party algorithm NAUTY for isomorphism discovery [32]. Here, we give a classic example in Fig. 2.

Algorithm 3 *ExtSubgraph*$(V_{Subgraph}, V_{ext}, v)$

if $|V_{Subgrqaph}| == k$ **then**
  output $G[V_{Subgraph}]$ and return
end if
while $V_{ext} \neq \emptyset$ **do**
  Remove an chosen node n from V_{ext} discretionarily
  $V'_{ext} = V_{ext} \cup \{u \in N_{excl}(n, V_{Subgraph}) : u > v\}$
  ExtSubgraph$(V_{Subgraph} \cup n, V'_{ext}, v)$
end while
return

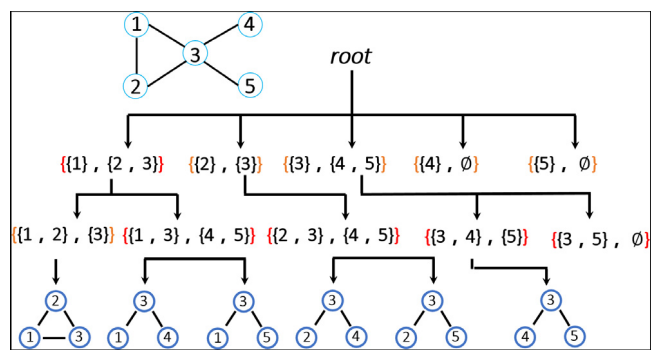

Fig. 2. Recursive search tree of the network: Given a five-node network, the iteration starts from random nodes with its N_{excl}. Finally, the subgraph result are shown below the search tree [29].

This method is able to detect specific subgraphs using unbiased subgraph sampling based on ESA. However, because of NAUTY, FanMod is also time-consuming.

4.3. Grochow

Grochow *et al.* proposed an algorithm for counting subgraphs in the networks [33]. Different from other network-centric methods introduced above, Grochow is a motif-centric approach [34]. This method focuses on the frequencies of the given non-

isomorphism subgraphs, which are called "query graph". The advantages of Grochow rely on preventing the ever increasing complexity of subgraph sampling. Instead of enumerating edges and nodes from the input graph, Grochow uses the mapping strategies called "symmetry-breaking". This shortens isomorphism computational time by avoiding multiple mapping operations.

The Grochow algorithm discovers the network motifs in two steps. First, it computes the symmetry-breaking constraint conditions of the query graph. Then it finds the possible mapping from the query graph to the input networks, which satisfies the associated symmetry-breaking conditions. Here, we give an example to explain the symmetry-breaking conditions in Fig. 3. The $Aut(G)$ needs to be first obtained to achieve the symmetry-breaking conditions of given networks G and the query graph. Although the graph automorphism problem is NP-complete, this method uses the McKay's tools and gtools package to settle this problem efficiently [32]. Grochow is depicted in Algorithm 4 [17].

Algorithm 4 *Grochow Algorithm*

Require: G, k and a query graph H
Ensure: All the appearances of H in G
1: H_E =EQUIVALENCEREPRESENTATIVES(H)
2: C =SYMMETRYBREAKINGCONDITIONS(H)
3: Sort $g \in V(G)$ by increasing degree and then by increasing neighbour degree sequence
4: **for** $v \in V(G)$ **do**
5: **for** $u \in H_E$ **do**
6: **if** SUPPORTS(g, h) **then**
7: f = partial map associating $f(u) = v$
8: ISOMORPHICEXTENSIONS(f, F, G, C)
9: **end if**
10: **end for**
11: **end for**
12: Remove g from G

Algorithm 5 *IsomorphicExtensions*(f, F, G, C)

1: D = domain of H
2: **if** D==H **then**
3: FoundOccurrence(f)
4: m = most constrained neighbours of any $d \in D$
5: **end if**
6: **for** $n \in N(f(D))$, with $d \in D$ **do**
7: **if** $f(m) = n$ does not violate C **then**
8: **if** $\nexists d \in N(m) : n \notin N(f(d))$ **then**
9: **if** $\nexists d \in N(m) : n \in N(f(d))$ **then**
10: $f' = fonD$ and $f'(m) = n$
11: *IsomorphicExtensions*(f, F, G, C)
12: **end if**
13: **end if**
14: **end if**
15: **end for**

In Algorithm 4, the step 8 is elaborated in Algorithm 5. In order to avoid unnecessary and redundant searches, the Grochow starts by detecting the equivalence classes of the query graph, which aims at representing each class mapping uniquely. According to a series of symmetry-breaking conditions in line 2, restrictions are added on the labeling of nodes to avoid symmetries. To make sure of that, all the equivalence classes are traversed, and the conditions are checked to guarantee that the node label from the input graph is the smallest one. After then, the procedure of Isomorphic Extensions starts by matching every node v from the graph G into one of the node u. Then the node v is searched by the degree to impose the constraints on the possible candidate neighbor nodes. Once there exists one node u satisfies the conditions, the degree and neighbor degrees of u are matched. Then, the algorithm recursively maps the nodes in the input networks to verify if all the query graph can be mapped. As it is described in Step 4 of Isomorphic Extensions, the neighbor nodes of v with the fewest possible candidates are selected. The candidate neighbor node satisfying the following two conditions will be added to the mapping recursively until all the query graph is discovered. (1) The candidate neighbor node does not break the existing symmetry conditions. (2) The candidate neighbor node does not induce a contradictory neighborhood. Finally, we can solve the isomorphism problem and detect all the subgraphs corresponding to the query graph. Furthermore, this algorithm includes a direct computation to distinguish whether the specific subgraph is a motif. The subgraphs sampling with the Grochow algorithm can be larger than those other algorithms. Moreover, the running time of this method is dramatically faster.

4.4. MODA

Omidi *et al.* proposed MODA for motif discovery in undirected networks [35]. It can be used in finding both induced and non-induced network motifs. Similar to Grochow, MODA is motif-centric. It is necessary to distinguish the motif-centric approaches due to their ability of finding motifs. The motif-centric mechanism discovering a single motif query or a small number of motif queries with large size rather than all possible subgraphs of a given size. Although motif-centric algorithms generally lack the accuracy in finding all possible large size subgraphs, they still perform well in discovering certain kinds of large size subgraphs.

MODA employs a hierarchical structure named "expansion tree" to extract network motifs of a given size. It avoids enumerating under-representing subgraphs. The tree expansion of this algorithm is used to calculate the subgraph frequency F_1. Herein, we discuss how it works in detail. Suppose $f(G)$ is a mapping of graph G with k nodes into the network, and G' is a graph contains one more edge (u, v) than G. f_G will map G' into the network if there exists an edge $(f_G(u), f_G(v))$ in the network. Then the mapping set of G' can be exploited to get the subgraph frequencies without any isomorphism computation, whose time complexity is only $O(1)$. MODA first finds the minimally connected query graphs of size k, as well as their mappings in the network based on subgraph isomorphism. Then, it expands the query graphs edge by edge and computes the frequency of these expanded graphs. The expansion procedure continues until the subgraph reaches a complete graph.

MODA can be implemented by expanding tree T_k for each k. The expansion tree T_k represents a directed acyclic graph. The root number k refers to the graph size existing in the expansion tree. Nodes in the first level of T_k give a restriction of size k, and the query tree expands with one edge at each level through traversing T_k in depth. The longest path in T_k contains $(k^2 - 3k + 4)/2$ edges. When the longest path is the only path from the root to the leaf node indicates the complete graph. Fig. 4 shows examples of 4-node query graphs. The expansion tree can be constructed in a simple way as introduced in [35].

MODA traverses T_k, then computes and saves the mapping set for the next step when it extracts query trees from the first level of T_k. As for non-tree queries from T_k, MODA extracts the mappings associated with the parent node in T_k and determines one of those mappings, which can support the current query graphs. The operation will not stop until it obtains the complete query graph. MODA innovates the sampling method in which the sampling of each node in the network is linearly proportional to the degree of nodes. Furthermore, the degree distribution is much similar to the Barabàsi–Albert preferential attachment model in complex networks area [36].

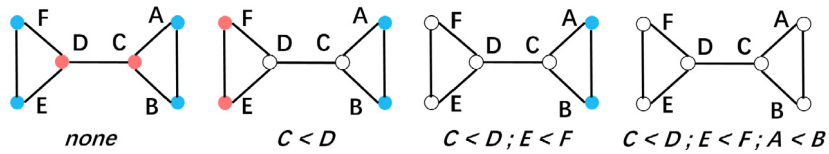

Fig. 3. An example of finding the symmetry-breaking constraint conditions on a graph consists of six nodes. Nodes in white are fixed by any automorphism preserving the indicated conditions, and other nodes are colored with shaded according to their equivalence class under the automorphisms which preserve the indicated conditions [33]. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

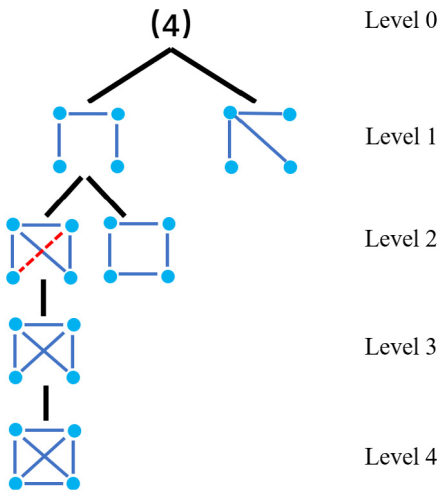

Fig. 4. The expansion tree T_4 for 4-node query graphs: there exist no isomorphic k-size trees at first. An edge is added to the parent graph in order to generate a child graph at each level. In level 2, the red dashed line indicates the existence of two alternative edges in the graph, which also represents the two expanded graph are isomorphic [35].

4.5. Kavosh

Kashani *et al.* proposed "Kavosh" to discover motifs in k-size with less memory and CPU time. Moreover, Kavosh can be applied no matter in directed or undirected networks [37]. The procedure of motif enumeration in Kavosh is similar to Grochow and MODA. This algorithm contains mainly four phases:

1. *Enumeration*: discovers all the k-subgraphs from the input networks;
2. *Classification*: classifies the discovered k-subgraphs into isomorphic groups;
3. *Random graphs generation*: generates random graphs based on the input network;
4. *Motif identification*: identifies the network motifs within the k-subgraphs, which have been found on the basis of the statistical definitions.

The *enumeration* subtask makes Kavosh algorithm different from other methods. Furthermore, *classification* is another vital subtask of this algorithm. NAUTY [32,38] algorithms are used in the *classification* phase to improve the efficiency of Kavosh.

In the *enumeration* phase, it first finds all the k-subgraphs and then removes the discovered nodes. Then, the algorithm repeats this procedure consecutively for all the remaining nodes. A searching tree is constructed for enumeration. To be more specific, for sampling the k-subgraphs that start with a particular node, the tree rooted at the given nodes and built on the neighborhood relationships having the maximum depth of the tree. The following steps expand a tree. First, select a child for each level, which satisfies that the child is not in any upper level. After reaching to the lowest level, the tree is traversed back again. The procedure is repeated with the stipulation. Specifically, the nodes that have been visited in previous paths of a descendant are now considered as undiscovered. One more restriction in constructing trees is that all the children in a particular tree must have larger numerical labels than that of the root node of the tree. The numerical label of each node in the graph is used to avoid over-counting subgraphs.

In order to extract k-subgraphs, all possible compositions of $k-1$ are considered. The compositions of $k-1$ comprise all possible ways to represent $k-1$ as the sum of positive integers. Summations with different summands' orders are distinctly considered. A composition can be presented as $k_2, k_3, \ldots, k_m$, satisfying $k_2 + k_3 + \cdots + k_m = k - 1$. To enumerate subgraphs according to the composition, k_i nodes are chosen from the ith level of the tree to be nodes of the subgraphs ($i = 2, 3, \ldots$). These chosen nodes as well as with the root node refine a subgraph. The detailed explanations about this procedure are illustrated in [37].

Comparing to other described algorithms used in both directed and undirected networks, Kavosh performs best in the aspects of running time and CPU memory cost. Meanwhile, Kavosh can be employed for discovering motifs of size greater than eight nodes.

4.6. G-tries

Pedro *et al.* demonstrate a *G-tries* based method for motif discovery [39]. The core idea of this algorithm is sampling the subgraphs by constructing a multi-way tree, which is akin to a prefix tree. This method makes use of the common topology structures of networks to construct a *g-tries*. The *g-tries* lists all the stored subgraphs, which include the occurrences of the given subgraphs in the input networks. Though all the subgraphs are derived from random networks, this process is time-consuming. Therefore, the *g-tries* structures can classify the subgraphs of random networks without using NAUTY. Hence, *g-tries* is the fastest algorithm on random networks.

As mentioned above, a *g-tries* is a multi-way tree, including a set of subgraphs. Each node contains the information of this node and its edges connecting to ancestor nodes. A path from the root node to a leaf in the tree corresponds to an individual graph. The descendants of a *g-tries* share the common subgraphs. The details about how to build a *g-tries* are described in [39]. Here, we briefly explain the *g-tries* with four-node networks in Fig. 5. The procedure for constructing a *g-tries* starts with an empty set that only contains one root node. The trees are traversed repetitively to verify whether the partial adjacency matrices row of any two children are the same when the new subgraphs are generated by insertions. Each node in the subgraph of the tree stores information about a new node and the connections to its ancestor. The corresponding row of the matrix represents each subgraph in a *g-tries*. If there exists a connection in the graph, we represent the graph with "1" in the matrix, otherwise "0". As we know, different adjacency matrices can represent the same graph since nodes can be labeled in different orders. Therefore, canonical labeling is used in this method to address the isomorphism problem. It guarantees that the isomorphism graphs can always generate the same adjacency matrices.

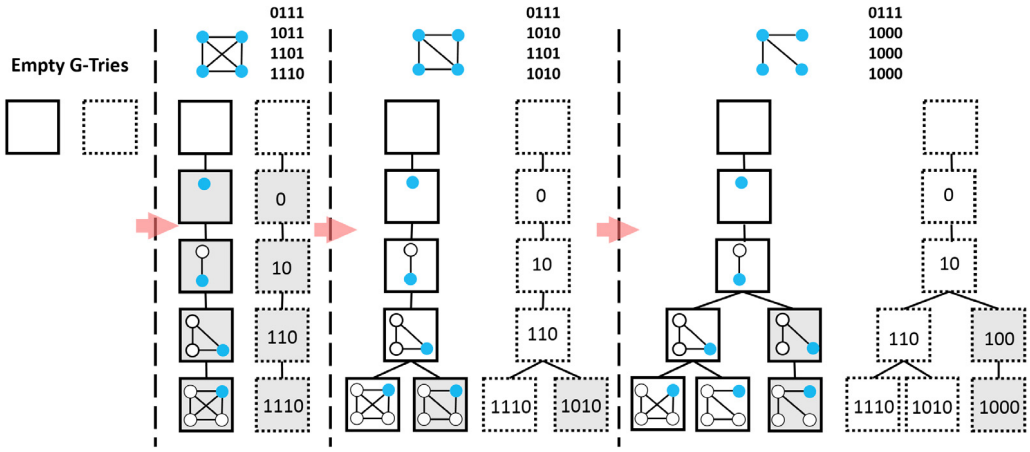

Fig. 5. The subgraphs in gray square are new ones after the insertion of nodes. And the black nodes indicate newly added nodes, while nodes in white are the remaining ones from before. And the squares with dashed lines are the actual *g-tries* implementations with adjacency matrices, the normal line squares is the correspondent visual representation of the subgraph. The matrices rows in this figure illustrate the connection of the new adding nodes (black nodes) [39].

The core idea of the sampling and counting process is to backtrack all possible subgraphs while computing isomorphism. This method takes advantage of the common structures of different candidate subgraphs, which saves much time in verifying isomorphism. There are no limitations in implementing this *g-tries* algorithm. However, this algorithm is still time-consuming on the subgraphs instead of input networks. Therefore, this algorithm is feasible when the target sampling subgraphs are in small size, (i.e., the elementary connected structures in the network). However, it is probably time-consuming or even infeasible when the sampling subgraphs are in large sizes. According to the experimental results, the sampling subgraphs can contain nine nodes at most [39].

4.7. QuateXelero

Khakabimamaghani *et al.* presented a motif discovery method named QuateXelero [40], which aims at simplifying the process of NAUTY. However, the core search tree structures are based on the ESU, which are the same as in FanMod. QuateXelero employs a quaternary tree structure based on ESU. A quaternary tree is a rooted tree structure with each internal node having four children at most. In other words, each internal node in the tree can have five neighbors, including its parents and children. Similar to *g-tries*, the quaternary tree also performs a partial classification when searching the subgraphs.

Similar to the existing motif discovery methods, QuateXelero has three main procedures: *enumeration*, *classification* and *detection* [40]. The usage of a quaternary tree in enumeration makes QuateXelero different from that in FanMod. In other words, as the partial subgraph is extended by one node, the quaternary tree is also searched some level further. The details of the enumeration phase are described in [40]. As for the classification, it is based on the rationale that if two different subgraphs reach the same leaf in the proposed quaternary tree, the two subgraphs are isomorphic to each other. However, the reverse does not establish, that means two isomorphic subgraphs may reach two different leaves of the tree. In that case, there may exist two or more different quaternary tree leaves pointing to the same binary tree leaf. Finally, some statistic parameters, such as *z-score*, are calculated to evaluate the network motifs.

Experimental results in [40] show that QuateXelero outperforms ESU and *g-tries* in finding the census on the original input networks. Nevertheless, it performs with lower efficiency on random networks for large-size motifs especially in directed networks. In directed networks, QuateXelero shows superiority in small and probably large motif detection instead of medium size motifs. Comparing to Kavosh and FanMod, QuateXelero performs better in the aspect of running time. Because the number of subgraphs is much larger than the number of non-isomorphic classes. In other words, a large number of subgraphs will reach the same leaf in the quaternary tree, so it is unnecessary to call the NAUTY except for the first one. Hence, the computational time of motif discovery is significantly reduced.

4.8. Color-coding approach

Most existing strategies in the field of motif discovery focus on finding induced subgraphs in the original networks. However, Noga Alon *et al.* presented a method to find non-induced subgraphs in undirected networks, e.g., protein–protein interaction (PPI) networks [41]. This approach counts non-induced trees and bounded tree width subgraphs. Additionally, it can calculate the subgraphs with less than ten nodes.

This method counts the number of non-induced occurrences by the following procedures:

1. *Color coding* : Paint each node in G randomly with one color from all k colors.
2. *Counting*: Enumerate the number of non-induced occurrences for T, wherein a unique color paints each node. Details can be looked up in [41].
3. *Iterating*: Carry out the above two steps repeatedly ($O(e^k)$ times) and calculate the sum number of the occurrences of T to obtain an estimate on the number of its occurrences in G.

On the account that the PPI networks are far from complete and error-free, this method is proper for network motif discovery for such networks. Both Grochow and Color-coding algorithms are suitable for non-induced subgraphs. However, the Color-coding algorithm performs better than the Grochow algorithm in terms of time complexity [41].

4.9. GPU-based approach

The QuateXelero algorithm computes the number of motifs in the networks, which requires a large number of motif isomorphism matching processes. However, isomorphism matching is proved to be *NP-hard* problem, which results in high computational cost. Authors in [42] proposed a method utilizing the

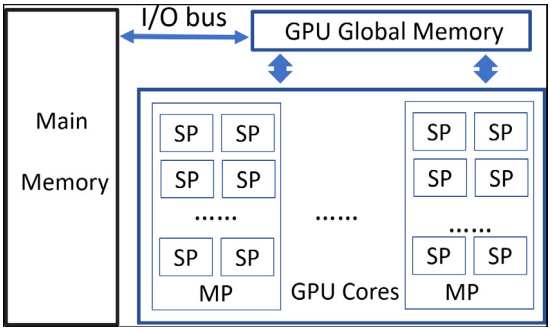

Fig. 6. Architecture of the GPU.

GPU (Graphics Processing Units) to parallelize a large number of isomorphism matching processes and apply this method to the directed unlabeled networks. The existing CPU-based subgraph isomorphism matching methods cannot be applied to GPU-based algorithm efficiently. Therefore, a novel subgraph isomorphism matching technique is proposed to compensate for the GPU-based motif discovery algorithm.

As Fig. 6 shows, a GPU contains a series of multiprocessors (*MP*), each of which is composed of numbers of stream processors (*SP*). In the *SP*s, the same instructions can be executed by the stream processors in parallel. Each MP in GPU has an individual cache. Furthermore, all the *SP*s of GPUs own the access to a larger but slower centralized memory. The input data can be exchanged between the cache of GPU and main memory through a high speed I/O bus. In this algorithm, the Nvidia CUDA programming framework is proposed to parallel computing on GPUs. This framework determines whether the codes run on a CPU or GPUs. Moreover, it contains four key concepts, i.e., *kernels*, *thread hierarchy*, *branch divergences*, and *memory coalescing*.

Each *kernel* in the CUDA framework starts by transferring the input data from the cache to the centralized memory of GPU, and then process the data on GPU. Then the results from the centralized memory are written back to the individual cache. However, if CPU and GPU are in the same memory, there is no need to be transferred. The GPU executes every kernel with a user-specified number of threads, which can be divided into several blocks corresponding to the *MP*. And each *MP* can be divided into smaller *waps*, which are executed concurrently. Because of the SIMD nature of *SP*s, all the threads on the GPU cannot execute different programs at the same time. Hence, if threads in the same *warp* have different execution paths, the threads can execute sequentially. *Memory coalescing* controls the data accessing among *warps* and *memory*. Each *MP* in GPU is responsible for a set of *warps*. In that case, *MP* controls the data retrieval and access. It also distributes the data to each thread, instead of respectively issuing one access for each thread. The *memory coalescing* reduces memory access overhead.

Here, we summarize the motif discovery algorithm proposed in [42] as shown in Algorithm 6. First, we define the M_s as a set containing a series of target motifs, which is represented as $M \in M_s$. $\mathcal{G}_r$ is a randomized network set, including r randomized networks, and $H \in \mathcal{G}_r$ is one of the randomized networks. The frequencies of M from M_s is calculated by Algorithm 6. In Algorithm 6, the vertices in H are arranged in ascending order $\langle v_1, v_2, \ldots, v_k \rangle$ for *isomorphism* verification. The isomorphic subgraph $M(2)$ with two nodes $\langle v_1, v_2 \rangle$ are detected and stored in R_2. Then, according to the vertices sequence, the isomorphic subgraph set R_i, $(i \in [3, k])$ is generated by the iterations from line 4 to 6. Finally, in Step 7, the algorithm returns the isomorphic subgraph set R_k, and the frequency of the given motif M equals to $|R_k|$.

Algorithm 6 *Motif Discovery*

Require: A motif $M \in M_s$, a randomized network $H \in \mathcal{G}_r$
Ensure: The frequency of M in H
1: [CPU]: Choose a matching order of the vertices in M, denoted as $< v_1, v_2, \ldots, v_k >$;
2: $M(2)$: the subgraph of M induced by $\{v_1, v_2\}$;
3: [CPU]: Identify the set R_2 of all the subgraphs in H which are isomorphic to M_2;
4: **for** $i = 3, \ldots, k$ do **do**
5: $M(i)$: the subgraph of M induced by $< v_1, v_2, \ldots, v_i >$;
6: [GPU]: According to R_{i-1}, calculate the set R_i of subgraphs in H, which are isomorphic to $M(i)$;
7: **end for**
8: return R_k

In contrast to other CPU-based methods [31,37,39,40,43–45], *Motif Discovery* does not construct any canonical labels or indices for the motifs in M_s. It helps to prevent the branch divergence and memory coalescing problems, which results in the inefficiency of the existing methods on GPUs. Therefore, an improved version is proposed in Algorithm 6. Firstly, the approach identifies the subgraphs of H, that is matching the subparts of motif M (i.e., $M(2)$, $M(3)$, $\ldots$, $M(k-1)$), and then pinpoints the k-motif isomorphic subgraph set with a series of "partial occurrences" of M. Although the improved approach is not as efficient as the CPU-based canonical labeling methods in determining whether the subgraph in G is isomorphic to the target graph g, it still takes advantage of GPUs.

All graphs in this algorithm are depicted in the boost website, which provides free peer-reviewed portable C++ source libraries.[1] To improve the calculating efficiency, six *edge arrays*, i.e., E_{out}, E_{in}, E_{bi}, O_{out}, O_{in}, and O_{bi} are used to implement the method on GPUs. The elements in the arrays are ordered pairs, for example, E_{out} is $\langle v_1, v_2 \rangle$, in which v_2 is an *out-neighbor* of v_1. Similar to E_{out}, E_{in} and E_{bi} of G correspond to the tuples of vertices in G with its *in-neighbor* or *bi-neighbor* vertices, respectively. O_{out}, O_{in}, and O_{bi} separately correspond to *offset arrays* of E_{out}, E_{in}, and E_{bi}. We regard that O_{out} is more than the tuples that map each vertex to a randomized network $H \in \mathcal{G}_r$, and the value that equals to the total number of elements in E_{out} plus one are appended in O_{out}. Accordingly, O_{in} and O_{out} are defined in the same way. Two more arrays, O_{nbr} and E_{nbr} are defined. For each vertex u in V, E_{nbr} keeps accounts of the set $nbr(u)$ and $u's$ neighbors. O_{nbr} gathers the position of the first neighbor of u in E_{nbr}. Moreover, the vertex set of $nbr(u)$ are kept consecutively in E_{nbr}, and the vertices are ordered by their IDs.

To implement the improved approach on a GPU, this method is illustrated as follows. First, the input graph $G = \{V, E\}$ is transferred to the GPU memory while stored with six arrays, E_{out}, E_{in}, E_{bi}, O_{in}, O_{out}, and O_{bi}. The two additional arrays, O_{nbr} and E_{nbr} are loaded into the GPU memory. Second, the procedure of enumerating the k-vertex combinations [42] with the incremental approach in GPU memory. According to the valid k-combinations in G, the adjacency matrices are computed and grouped to derive the unique adjacency matrices set $\mathcal{M}_k$ on GPU. Then the canonical labeling for each adjacency matrix is computed in parallel after transferring $\mathcal{M}_k$ to CPU memory.

5. Motif discovery tools and platforms

In the motif discovery process, some tools and platforms are effective to analyze and visualize the data. In this part, we introduce several motif discovery tools and platforms, as well as popular network analysis and visualization tools.

[1] http://www.boost.org/doc/libs.

5.1. Motif discovery tools

5.1.1. MFinder

MFinder is a motif detection and analysis tool [27]. The core idea has been described in Section 4.1. This tool is available for both Windows and Linux system and has been upgraded to version 1.2 that is open-sourced. It is worth to mention that MFinder cannot be carried on visualization but based on the visualization tool named mDraw,[2] which can also be downloaded from the same URL as MFinder.

5.1.2. FanMod

FanMod is an open-source and fast network motif detection tool [31]. It is available for Windows, Linux, and Mac OS. FanMod can detect motifs with 8 vertices and colored them with 7 colors at most. It implements the algorithm RAND-ESU to enumerate subgraphs [46]. Moreover, RAND-ESU can efficiently process a big network. Also, FanMod can analyze colored networks, as well as export results in CSV or HTML format. Furthermore, FanMod can be applied to detect both induced and non-induced subgraphs.

5.1.3. Regmex

Regmex offers several methods to identify overrepresented motif in a ranked list of sequences [47]. It implements Markov models to calculate probabilities for motif detection in the sequence and it uses random walks, Brownian bridges, or modified rank-based statistical methods to evaluate motif frequency. With a list of sequences (ranked by the arranged settings) and one or more input motifs, Regmex generates the rank correlation or clustering *p-values* of the motifs. Regmex is widely used for biological network motif discovery.

5.1.4. RSAT peak-motifs

RSAT peak-motifs is a computational pipeline that supports the motif discovery motif in peak sequences. It presents a user-friendly web interface and provides a full analysis of ChIP-seq peaks or similar data without any restriction on sequence size or peaks. It takes peak sequences as input, discovers higher frequency motifs, compares them with databases, and predicts binding site positions. Finally, it returns an HTML report and visualization link in the UCSC genome browser.

5.1.5. STAMP

STAMP is a platform for studying DNA-binding motifs [48]. It is a web server that contains various services for DNA-binding frequency motifs. It supports the formats of regular motif, frequency matrix, as well as the uploading of 12 motif-finders output files.

5.1.6. MotifNet

MotifNet is a new network motif analysis tool [49]. Based on FanMod [31], which is the first open-access web-server. MotifNet can analyze integrated networks and search for motifs of up to eight nodes. Users can also distinguish node-centric motifs and those recur in different parts of network components. The output motifs are visualized with filters, which enables users to filter out unnecessary information. Also, MotifNet provides a quantitative measurement for the motif instances in the network.

5.2. Network analysis and visualization tools

5.2.1. Gephi

Gephi is an open-source software for complex network visualization and analysis. Commonly, it is used for data analysis, link analysis, and network analysis. Users can upload data from files, databases, Emails, websites, and generated data based on their demand. Additionally, Gephi allows users to customize the layouts, metrics, data source, etc. by various plugins.

5.2.2. NodeXL

NodeXL is a plugin and template for Microsoft Excel(version 2007 or higher). With NodeXL, users can analyze and visualize the complex networks with mouse clicks as simple as Excel. It also allows users to filter and customize the visualization results with a few steps. NodeXL basic is an open-source tool, while NodeXL Pro is a commercial version that provides some additional features including easy access to social media network data streams, advanced network metrics, text and sentiment analysis, and powerful report generation.

5.2.3. Pajek

Pajek is a powerful analysis and visualization tool for large and complex networks. It supports windows platform and outperforms other methods when processing networks with millions of nodes. Pajek offers a step-by-step tutorial which facilitates users to learn and use it. It also supports the *R* interface that extends its statistic functions.

5.2.4. GUESS

GUESS is another analysis and visualization tool for graphs and networks. Users can import data with standard formats (Pajek, GML, etc.), and export various image types, and videos for dynamic and time-sensitive data. GUESS is based on Gython and Java, and users use Gython, a domain-specific embedded language derived from Python to work on graph structures.

6. Applications and challenges of motif discovery

Motif discovery task plays a vital role in network science as well as other research fields. Network motifs provide new insights for scholars to study networks from macroscopic, mesoscopic, and microscopic perspectives [3]. Hence, they are significant for numerous applications. According to the motif distributions of networks, we can characterize a network from another perspective. In this section, we briefly illustrate the applications of network discovery and their challenges.

- **Large networks comparison and classification:** Motifs can be used in handling network comparison and classification tasks. This enables networks similarity being measured by the statistical distribution of feature-based motifs. The notion of isomorphism equivalence to some forms of *structural similarity* among loose networks.
- **Discovering special patterns of networks:** The problem of finding a special structure, such as cliques, stars, etc., is NP-hard [50]. The motifs in networks are elementary structures in large-scale networks. Small motifs are likely to be part of stars or cliques. Therefore, special patterns can be regarded as a combination of motifs.

At present, new technologies have enabled the visualization and analysis of networks containing millions of nodes. Abundant motif discovery algorithms have been presented accordingly and applied in various research areas. However, there are still challenges in the motif discovery domain.

- **Computational complexity:** Though approaches (e.g., GPU-based approaches) are proposed to accelerate motif discovery algorithms, the computational complexity of these algorithms is still high. Comparing with other relative network-based algorithms, motif discovery algorithms are still time-consuming. Essentially, as motif discovery is a subgraph finding problem, the running time is exponentially increasing with the size of the network. Therefore, further work should put more efforts in order to rise to this challenge.

[2] http://www.weizmann.ac.il/mcb/UriAlon/.

- **Higher order motif discovery:** Motif discovery has practical applications in various areas. Most of motif discovery algorithms generally focus on motifs in small sizes. However, higher order motifs are significant. Many works of literature mentioned that higher order motifs have realistic meanings and can be applied potentially in many issues. For example, teams in academic social networks can be regarded as a higher order motifs. This kind of higher order motif organization has become important along with an extremely enlarged network scale. Therefore, it is both significant and high-demanding to discover higher order motifs in larger structures.
- **Finding the same motifs:** Finding the same motifs differs from finding homogeneous motifs. It is another technical challenge of motif discovery. This is because that same motifs need to be distinguished from homogeneous motifs in a mapping process, which is time-consuming. This process needs both high accuracy and complete coverage in the whole network. Therefore, in order to handle this challenge, the computational complexity should be handled first.
- **Discovering heterogeneous sizes of motifs:** Heterogeneous networks have a large occupation in a realistic world. Existing methods rarely focus on discovering heterogeneous sizes of motifs due to its complexity and heterogeneity. Nevertheless, strategies on finding heterogeneous sizes of motifs are even more meaningful than homogeneous sizes of motifs in many fields, especially in social science, network science, etc. Meanwhile, heterogeneous sizes of motifs are more difficult to be discovered in networks. The technical difficulties of discovering heterogeneous sizes of motifs need to be overcome further.
- **Visualizing motif discovery results:** Motif discovery has shifted its original applications from biological field to network science domain. Existing visualization tools are mostly developed for biological information networks. The visualization results are significantly different but essential in many other research fields. As a result, motif visualization has become a trending research area gradually.

7. Conclusion

Networks are widely used to model scientific issues in various disciplines, wherein network motif discovery is significant and of great value. In this work, we discuss the network motif discovery problem, correspondingly, motifs discovery algorithms. We summarize motif discovery algorithms from several perspectives, including theoretical basis, algorithm flows, scopes of application, as well as relative metrics. We also introduce the tools and platforms for motif discovery and present future challenges as well. More efforts should be put on reducing computing complexity, discovering higher order motifs, finding the same motifs, and discovering heterogeneous sizes of motifs. In summary, motif discovery is still an open research issue and is a promising research field that needs more attention.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Fundamental Research Funds for the Central Universities (No. DUT16RC(4)63), and the National Natural Science Foundation of China (No. 61876031 and 61906028).

References

[1] Z. Wang, Y. Zhang, Y. Li, Q. Wang, F. Xia, Exploiting social influence for context-aware event recommendation in event-based social networks, in: IEEE INFOCOM 2017 - IEEE Conference on Computer Communications, 2017, pp. 1–9.
[2] D.A. Gianetto, M. Mosleh, B. Heydari, Dynamic structure of competition networks in affordable care act insurance market, IEEE Access 6 (2018) 12700–12709.
[3] N.K. Ahmed, J. Neville, R.A. Rossi, N.G. Duffield, T.L. Willke, Graphlet decomposition: Framework, algorithms, and applications, Knowl. Inf. Syst. 50 (3) (2017) 689–722.
[4] O.N. Yaveroglu, N. Maloddognin, D. Davis, Z. Levnajic, V. Janjic, R. Karapandza, A. Stojmirovic, N. Przulj, Revealing the hidden language of complex networks, Sci. Rep. 4 (2013) 4547.
[5] X. Kong, Y. Shi, S. Yu, J. Liu, F. Xia, Academic social networks: Modeling, analysis, mining and applications, J. Netw. Comput. Appl. 132 (2019) 86–103.
[6] R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, U. Alon, Network motifs: Simple building blocks of complex networks, Science 298 (5594) (2002) 824–827.
[7] M. Al Hasan, Methods and applications of network sampling, in: Optimization Challenges in Complex, Networked and Risky Systems, INFORMS, 2016, pp. 115–139.
[8] S. Boccaletti, V. Latora, Y. Moreno, M. Chavez, D.-U. Hwang, Complex networks: Structure and dynamics, Phys. Rep. 424 (4–5) (2006) 175–308.
[9] S. Mangan, U. Alon, Structure and function of the feed-forward loop network motif., Proc. Natl. Acad. Sci. USA 100 (21) (2003) 11980–11985.
[10] A.R. Benson, C. Riquelme, S. Schmit, Learning multifractal structure in large networks, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2014, pp. 1326–1335.
[11] P. Holme, J. Saramaki, Temporal Networks as a Modeling Framework, Springer Berlin Heidelberg, 2013, pp. 1–14.
[12] C.J. Honey, R. Kotter, M. Breakspear, O. Sporns, Network structure of cerebral cortex shapes functional connectivity on multiple time scales, Proc. Natl. Acad. Sci. USA 104 (24) (2007) 10240–10245.
[13] C. Lee, M. Pham, M.K. Jeong, D. Kim, D.K. Lin, W.A. Chavalitwongse, A network structural approach to the link prediction problem, INFORMS J. Comput. 27 (2) (2015) 249–267.
[14] Y. Yang, B. Padmanabhan, H. Liu, X. Wang, Discovery of periodic patterns in sequence data: A variance-based approach, INFORMS J. Comput. 24 (3) (2012) 372–386.
[15] Y. Hulovatyy, H. Chen, T. Milenkovic, Exploring the structure and function of temporal networks with dynamic graphlets., Bioinformatics 32 (15) (2016) 2402.
[16] J. Hu, R. Cheng, K.C.-C. Chang, A. Sankar, Y. Fang, B.Y. Lam, Discovering maximal motif cliques in large heterogeneous information networks, in: 2019 IEEE 35th International Conference on Data Engineering (ICDE), IEEE, 2019, pp. 746–757.
[17] P. Ribeiro, F. Silva, M. Kaiser, Strategies for network motifs discovery, in: 2009 Fifth IEEE International Conference on E-Science, IEEE, 2009, pp. 80–87.
[18] T. Swati, D. Soumitra, Efficient motif discovery, Int. J. Innov. Adv. Comput. Sci. 7 (3) (2018) 806–808.
[19] N. Pržulj, D.G. Corneil, I. Jurisica, Modeling interactome: Scale-free or geometric? Bioinformatics 20 (18) (2004) 3508–3515.
[20] R. Milo, S. Itzkovitz, N. Kashtan, R. Levitt, S. Shen-Orr, I. Ayzenshtat, M. Sheffer, U. Alon, Superfamilies of evolved and designed networks, Science 303 (5663) (2004) 1538–1542.
[21] H.A. Soufiani, E. Airoldi, Graphlet decomposition of a weighted network, in: Artificial Intelligence and Statistics, 2012, pp. 54–63.
[22] J. Luo, L. Ding, C. Liang, N.H. Tu, An efficient network motif discovery approach for co-regulatory networks, IEEE Access 6 (2018) 14151–14158.
[23] S.S. Shen-Orr, R. Milo, S. Mangan, U. Alon, Network motifs in the transcriptional regulation network of escherichia coli, Nature Genet. 31 (1) (2002) 64–68.
[24] V. Lacroix, C.G. Fernandes, M.-F. Sagot, Motif search in graphs: Application to metabolic networks, IEEE/ACM Trans. Comput. Biol. Bioinform. (TCBB) 3 (4) (2006) 360–368.
[25] L. Parida, Discovering topological motifs using a compact notation, J. Comput. Biol. 14 (3) (2007) 300–323.
[26] N. Kashtan, S. Itzkovitz, R. Milo, U. Alon, Efficient sampling algorithm for estimating subgraph concentrations and detecting network motifs, Bioinformatics 20 (11) (2004) 1746–1758.
[27] N. Kashtan, S. Itzkovitz, R. Milo, U. Alon, Mfinder Tool Guide, Technical report, Department of Molecular Cell Biology and Computer Science and Applied Mathematics, Weizman Institute of Science, Israel, 2002.
[28] R. Rane, Frequent subpatterns distribution in social network analysis, in: Emerging Technologies in Data Mining and Information Security, Springer Singapore, 2019, pp. 393–403.

[29] S. Wernicke, Efficient detection of network motifs, IEEE/ACM Trans. Comput. Biol. Bioinform. 3 (4) (2006).
[30] Y. Chen, Y. Chen, An efficient sampling algorithm for network motif detection, J. Comput. Graph. Statist. 27 (3) (2018) 503–515.
[31] S. Wernicke, F. Rasche, Fanmod: A tool for fast network motif detection, Bioinformatics 22 (9) (2006) 1152–1153.
[32] H. Wang, N. Li, J. Li, H. Gao, Parallel algorithms for flexible pattern matching on big graphs, Inform. Sci. 436 (2018).
[33] J.A. Grochow, M. Kellis, Network motif discovery using subgraph enumeration and symmetry-breaking, in: Annual International Conference on Research in Computational Molecular Biology, Springer, 2007, pp. 92–106.
[34] A. Masoudi-Nejad, F. Schreiber, Z.R.M. Kashani, Building blocks of biological networks: A review on major network motif discovery algorithms, IET Syst. Biol. 6 (5) (2012) 164–174.
[35] S. Omidi, F. Schreiber, A. Masoudi-Nejad, Moda: An efficient algorithm for network motif discovery in biological networks, Genes Genet. Syst. 84 (5) (2009) 385–395.
[36] S. Fortunato, C.T. Bergstrom, K. Börner, J.A. Evans, D. Helbing, S. Milojević, A.M. Petersen, F. Radicchi, R. Sinatra, B. Uzzi, et al., Science of science, Science 359 (6379) (2018) eaao0185.
[37] Z.R.M. Kashani, H. Ahrabian, E. Elahi, A. Nowzari-Dalini, E.S. Ansari, S. Asadi, S. Mohammadi, F. Schreiber, A. Masoudi-Nejad, Kavosh: A new algorithm for finding network motifs, BMC Bioinformatics 10 (1) (2009) 318.
[38] J. Chen, W. Hsu, M.L. Lee, S.-K. Ng, Nemofinder: Dissecting genome-wide protein-protein interactions with meso-scale network motifs, in: Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2006, pp. 106–115.
[39] P. Ribeiro, F. Silva, G-tries: An efficient data structure for discovering network motifs, in: Proceedings of the 2010 ACM Symposium on Applied Computing, ACM, 2010, pp. 1559–1566.
[40] S. Khakabimamaghani, I. Sharafuddin, N. Dichter, I. Koch, A. Masoudi-Nejad, Quatexelero: An accelerated exact network motif detection algorithm, PLoS One 8 (7) (2013) e68073.
[41] N. Alon, P. Dao, I. Hajirasouliha, F. Hormozdiari, S.C. Sahinalp, Biomolecular network motif counting and discovery by color coding, Bioinformatics 24 (13) (2008) i241–i249.
[42] W. Lin, X. Xiao, X. Xie, X.-L. Li, Network motif discovery: A GPU approach, IEEE Trans. Knowl. Data Eng. 29 (3) (2017) 513–528.
[43] X. Li, D.S. Stones, H. Wang, H. Deng, X. Liu, G. Wang, Netmode: Network motif detection without nauty, PLoS One 7 (12) (2012) e50093.
[44] P. Ribeiro, F. Silva, L. Lopes, Parallel discovery of network motifs, J. Parallel Distrib. Comput. 72 (2) (2012) 144–154.
[45] T. Wang, J.W. Touchman, W. Zhang, E.B. Suh, G. Xue, A parallel algorithm for extracting transcriptional regulatory network motifs, in: Fifth IEEE Symposium on Bioinformatics and Bioengineering (BIBE'05), IEEE, 2005, pp. 193–200.
[46] S. Wernicke, A faster algorithm for detecting network motifs, in: International Workshop on Algorithms in Bioinformatics, Springer, 2005, pp. 165–177.
[47] M.M. Nielsen, P. Tataru, T. Madsen, A. Hobolth, J.S. Pedersen, Regmex, motif analysis in ranked lists of sequences, 2016, p. 035956, http://dx.doi.org/10.1101/035956, BioRxiv.
[48] S. Mahony, P.V. Benos, Stamp: a web tool for exploring dna-binding motif similarities, Nucleic Acids Res. 35 (suppl_2) (2007) W253–W258.
[49] I.Y. Smoly, E. Lerman, M. Ziv-Ukelson, E. Yeger-Lotem, Motifnet: A web-server for network motif analysis, Bioinformatics 33 (12) (2017) 1907–1909.
[50] S.U. Rehman, S. Asghar, S.J. Fong, Optimized and frequent subgraphs: How are they related? IEEE Access 6 (2018) 37237–37249.